

Курсовая работа по дисциплине «Основы программирования» тема: Splay дерево

Автор работы:

Тутов Данил Андреевич, студент ПВ-211

Руководитель:

Притчин Иван Сергеевич,
старший преподаватель кафедры ПОВТАС



Цели

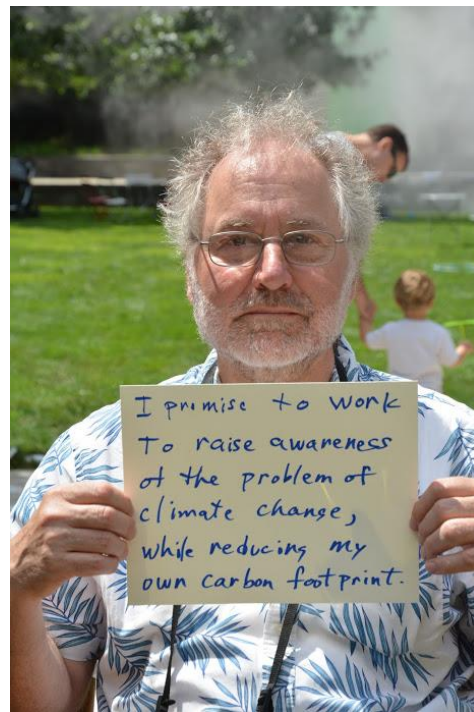
- 🔑 Изучить принципы структуры данных «Splay tree»
- 🔑 Реализовать операции для взаимодействия
- 🔑 Проанализировать сложность алгоритмов структуры



Введение



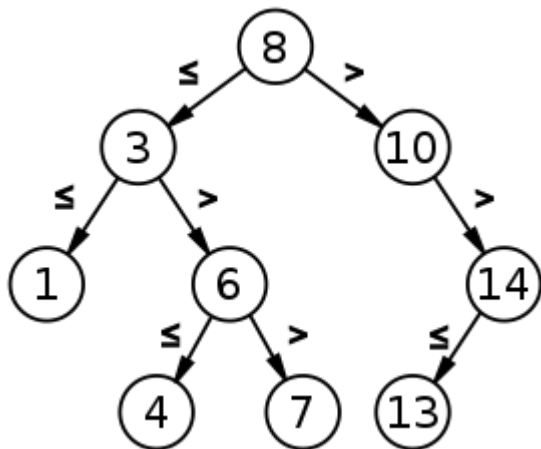
Роберт Тарьян



Даниэль Слейтор



Binary search tree (BST)

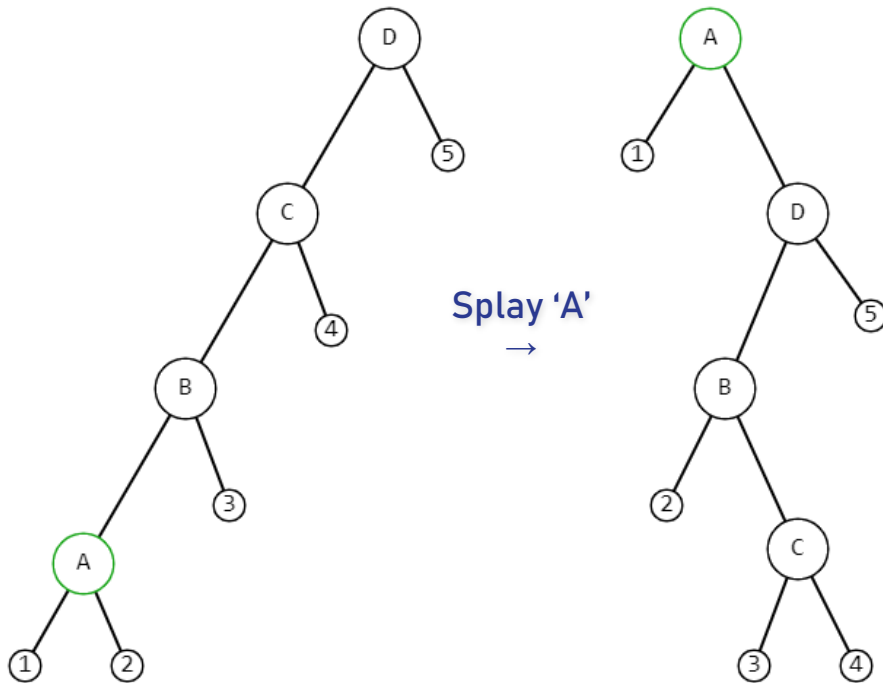


Сложность в O-символике

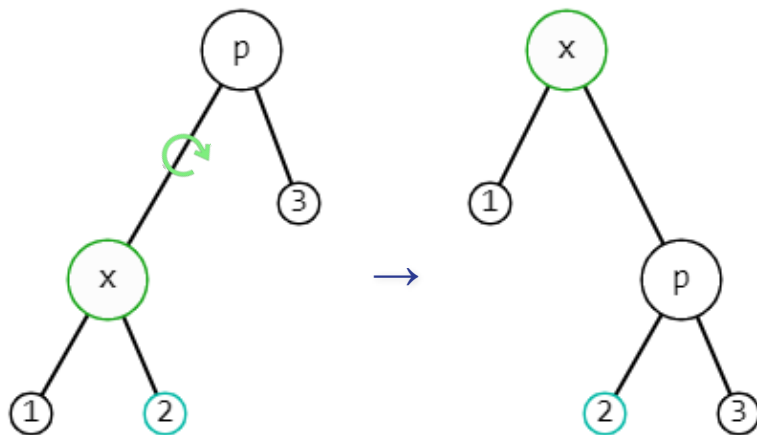
	В среднем	В худшем случае
Расход памяти	$O(n)$	$O(n)$
Поиск	$O(\log n)$	$O(n)$
Вставка	$O(\log n)$	$O(n)$
Удаление	$O(\log n)$	$O(n)$

Splay tree

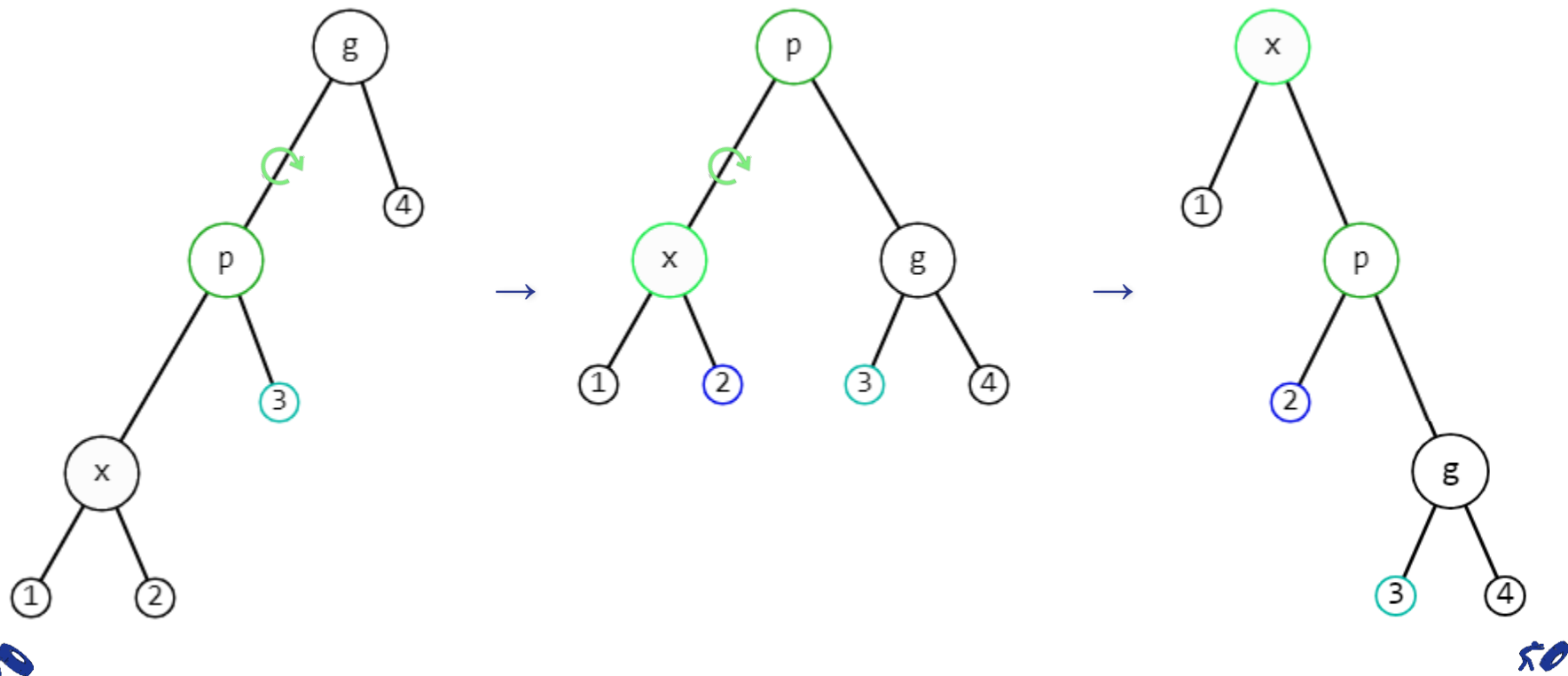
расширяющееся, косое,
растянутое,
выворачивающееся,
дерево отображения,
развёрнутое



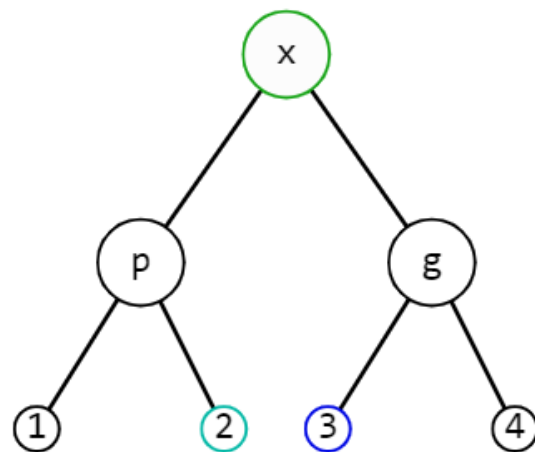
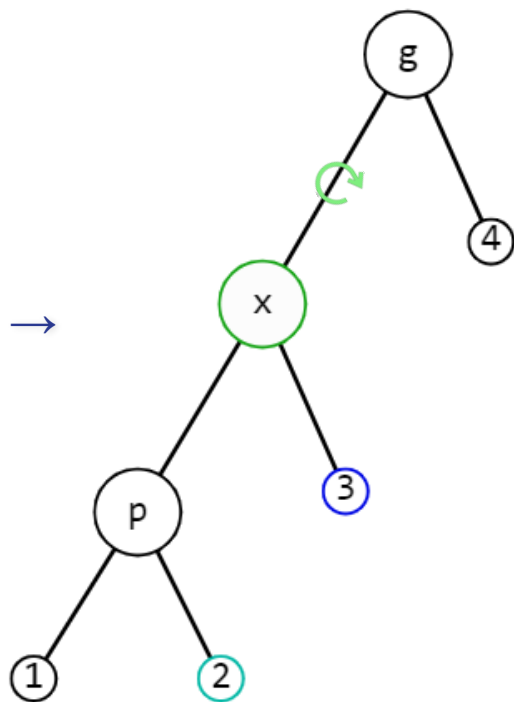
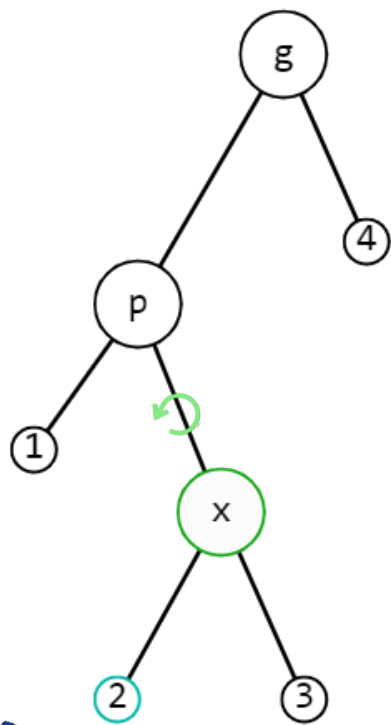
Операция Splay (случай zig)



Операция Splay (случай zig-zig)

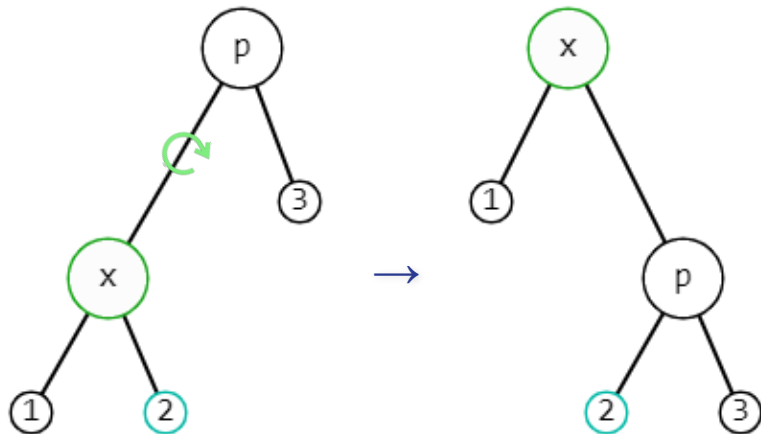


Операция Splay (случай zig-zag)



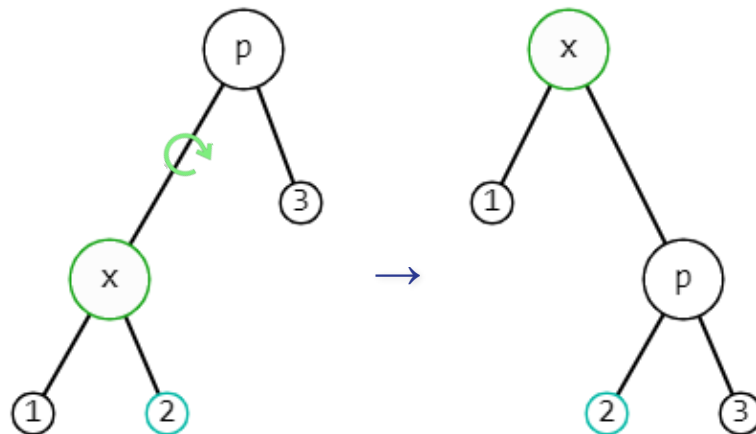
Операция rightRotate

```
1 // performs right rotation (rotation of the edge connecting n and it's left child)
2 void rightRotate(Node *n) {
3     Node *leftChild = n->leftChild;
4
5     // execute shift right child of the n's left child to n's left child
6     // (even if equal nullptr)
7     n->leftChild = leftChild->rightChild;
8     if (leftChild->rightChild) {
9         leftChild->rightChild->parent = n;
10    }
11
12    updateParents( parent: n, child: leftChild);
13
14    // link old child and old parent
15    leftChild->rightChild = n;
16    leftChild->rightChild->parent = leftChild;
17 }
```



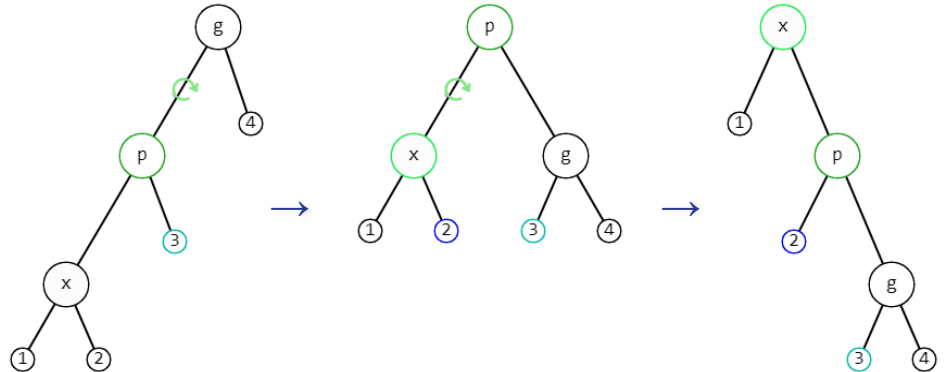
Операция Splay

```
1 // performs splay (delivers a node n to the root of the tree using left and right pivots)
2 void splay(Node *n) {
3     while (parent(n)) {
4         // case: node n is left child
5         if (n == parent(n)->leftChild) {
6             if (!grandparent(n)) { // there isn't grandparent
7                 // zig operation
8                 rightRotate(n: parent(n));
9             } else if (grandparent(n)->leftChild == parent(n)) { // n's parent - left child
10                // zig-zig operation
11                rightRotate(n: grandparent(n));
12                rightRotate(n: parent(n));
13            } else { // n's parent - right child
14                // zig-zag operation
15                rightRotate(n: parent(n));
16                leftRotate(n: parent(n));
17            }
18        } else { // case: node n is right child
19            if (!grandparent(n)) { // there isn't grandparent
20                // zig operation
21                leftRotate(n: parent(n));
22            } else if (grandparent(n)->rightChild == parent(n)) { // n's parent - right child
23                // zig-zig operation
24                leftRotate(n: grandparent(n));
25                leftRotate(n: parent(n));
26            } else { // n's parent - left child
27                // zig-zag operation
28                leftRotate(n: parent(n));
29                rightRotate(n: parent(n));
30            }
31        }
32    }
33 }
```



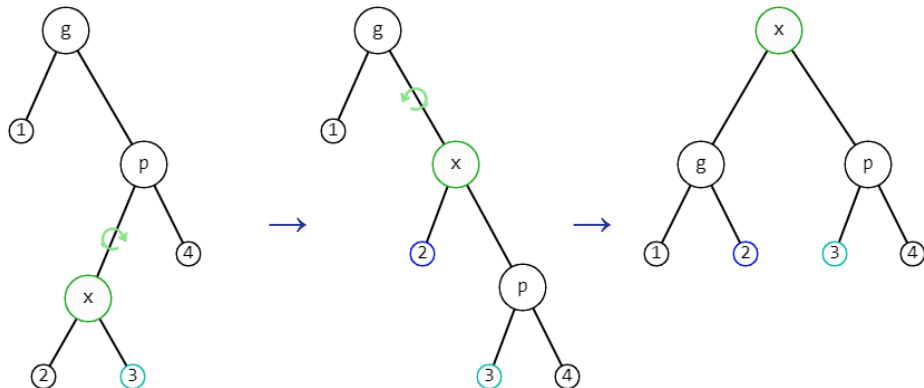
Операция Splay

```
1 // performs splay (delivers a node n to the root of the tree using left and right pivots)
2 void splay(Node *n) {
3     while (parent(n)) {
4         // case: node n is left child
5         if (n == parent(n)->leftChild) {
6             if (!grandparent(n)) { // there isn't grandparent
7                 // zig operation
8                 rightRotate(n: parent(n));
9             } else if (grandparent(n)->leftChild == parent(n)) { // n's parent - left child
10                // zig-zig operation
11                rightRotate(n: grandparent(n));
12                rightRotate(n: parent(n));
13            } else { // n's parent - right child
14                // zig-zag operation
15                rightRotate(n: parent(n));
16                leftRotate(n: parent(n));
17            }
18        } else { // case: node n is right child
19            if (!grandparent(n)) { // there isn't grandparent
20                // zig operation
21                leftRotate(n: parent(n));
22            } else if (grandparent(n)->rightChild == parent(n)) { // n's parent - right child
23                // zig-zig operation
24                leftRotate(n: grandparent(n));
25                leftRotate(n: parent(n));
26            } else { // n's parent - left child
27                // zig-zag operation
28                leftRotate(n: parent(n));
29                rightRotate(n: parent(n));
30            }
31        }
32    }
33 }
```



Операция Splay

```
1 // performs splay (delivers a node n to the root of the tree using left and right pivots)
2 void splay(Node *n) {
3     while (parent(n)) {
4         // case: node n is left child
5         if (n == parent(n)->leftChild) {
6             if (!grandparent(n)) { // there isn't grandparent
7                 // zig operation
8                 rightRotate(n: parent(n));
9             } else if (grandparent(n)->leftChild == parent(n)) { // n's parent - left child
10                // zig-zig operation
11                rightRotate(n: grandparent(n));
12                rightRotate(n: parent(n));
13            } else { // n's parent - right child
14                // zig-zag operation
15                rightRotate(n: parent(n));
16                leftRotate(n: parent(n));
17            }
18        } else { // case: node n is right child
19            if (!grandparent(n)) { // there isn't grandparent
20                // zig operation
21                leftRotate(n: parent(n));
22            } else if (grandparent(n)->rightChild == parent(n)) { // n's parent - right child
23                // zig-zig operation
24                leftRotate(n: grandparent(n));
25                leftRotate(n: parent(n));
26            } else { // n's parent - left child
27                // zig-zag operation
28                leftRotate(n: parent(n));
29                rightRotate(n: parent(n));
30            }
31        }
32    }
33 }
```



Базовые операции

 Вставка

 Поиск

 Удаление

 Объединение

 Разделение

```
// inserts an element with data insertingData
void insert(const T &insertingData) {
    Node *insertPlace = root;
    Node *parentOfInsertedPlace = nullptr;

    // finding suitable place
    while (insertPlace) {
        // necessary save previous insertPlace (it will be a parent of the insertPlace)
        parentOfInsertedPlace = insertPlace;
        if (insertingData > insertPlace->data) {
            insertPlace = insertPlace->rightChild;
        } else {
            insertPlace = insertPlace->leftChild;
        }
    }

    // creating inserting element
    Node *insertedNode = new Node(insertingData);
    insertedNode->parent = parentOfInsertedPlace;

    // link inserted element's parent with him
    if (!parentOfInsertedPlace) {
        root = insertedNode;
    } else if (parentOfInsertedPlace->data < insertedNode->data) {
        parentOfInsertedPlace->rightChild = insertedNode;
    } else {
        parentOfInsertedPlace->leftChild = insertedNode;
    }

    splay(n: insertedNode);
}
```

Базовые операции

 Вставка

 Поиск

 Удаление

 Объединение

 Разделение

```
// returns a pointer to the node with data searchingData
```

```
Node* find(const T &searchingData) {  
    Node* findingNode = root;  
  
    while (findingNode) {  
        if (searchingData > findingNode->data) {  
            findingNode = findingNode->rightChild;  
        } else if (searchingData < findingNode->data) {  
            findingNode = findingNode->leftChild;  
        } else {  
            splay(n: findingNode);  
            return findingNode;  
        }  
    }  
  
    return nullptr;  
}
```

Базовые операции

 Вставка

 Поиск

 Удаление

 Объединение

 Разделение

// removes an element with data deletingData

```
void erase(const T &deletingData) {
    Node* deletingNode = find( searchingData: deletingData);

    if (deletingNode) {
        if (deletingNode->rightChild == nullptr) {
            updateParents( parent: deletingNode, child: deletingNode->leftChild);
        } else if (deletingNode->leftChild == nullptr) {
            updateParents( parent: deletingNode, child: deletingNode->rightChild);
        } else { // case: there are two children owned deletingNode
            Node* replacementRemotedNode = _maximum( n: deletingNode->leftChild);

            // case: replacing node can have subtrees
            if (parent( n: replacementRemotedNode) != deletingNode) {
                updateParents( parent: replacementRemotedNode, child: replacementRemotedNode->leftChild);

                replacementRemotedNode->leftChild = deletingNode->leftChild;
                replacementRemotedNode->leftChild->parent = replacementRemotedNode;
            }

            updateParents( parent: deletingNode, child: replacementRemotedNode);

            replacementRemotedNode->rightChild = deletingNode->rightChild;
            deletingNode->rightChild->parent = replacementRemotedNode;
        }

        if (parent( n: deletingNode)) {
            splay( n: parent( n: deletingNode));
        }
        delete deletingNode;
    }
}
```

Базовые операции

 Вставка

 Поиск

 Удаление

 Объединение

 Разделение

```
// merges this tree with tree addedTree
void merge(const SplayTree &addedTree) {
    // necessary make sure that all keys of the current (this) tree less than the keys of the tree T2
    // and remember "smallest" tree
    SplayTree smallest, largest;
    if (this->root->data > addedTree.root->data) {
        smallest.root = addedTree.root;
        largest.root = this->root;
    } else {
        smallest.root = this->root;
        largest.root = addedTree.root;
    }

    smallest.splay(n: _maximum(n: smallest.root));

    smallest.root->rightChild = largest.root;
    this->root = smallest.root;
}
```


Базовые операции

 Вставка

 Поиск

 Удаление

 Объединение

 Разделение

```
// splits the tree into two trees
// first tree going to be in "this" tree, second in secondTree
// first tree contains elements less or equal than splitElement
// second tree contains elements greater than x
// note: not necessary splitElement there's in the tree
void split(SplayTree &secondTree, const T &splitElement) {
    Node* similarOrEqualToSplitElement = root;

    // finding element, which similar to splitElement (even can be equal)
    while (similarOrEqualToSplitElement) {
        if (splitElement < similarOrEqualToSplitElement->data && similarOrEqualToSplitElement->leftChild) {
            similarOrEqualToSplitElement = similarOrEqualToSplitElement->leftChild;
        } else if (splitElement > similarOrEqualToSplitElement->data && similarOrEqualToSplitElement->rightChild) {
            similarOrEqualToSplitElement = similarOrEqualToSplitElement->rightChild;
        } else {
            break;
        }
    }

    splay(n: similarOrEqualToSplitElement);

    secondTree.root = similarOrEqualToSplitElement->rightChild;
    similarOrEqualToSplitElement->rightChild = nullptr;
}
```

Заключение

- 🔑 Изучил принципы структуры данных «Splay tree»
- 🔑 Реализовал операции для взаимодействия
- 🔑 Проанализировал сложность алгоритмов структуры